

For simplicity, suppose that the mesh has disk topology, meaning that it can be mapped to the interior of a circle in the plane, and that we have fixed the location of each vertex on its boundary $B \subseteq V$. The job of the parameterization algorithm then is to fill in positions for the interior vertices of the mesh. This setup and the output of the algorithm outlined below are shown in Figure 4.5(b).

For a vertex $v \in V$, take $N(v)$ to be the set of neighbors of v on the mesh, given by

$$N(v) \equiv \{w \in V : (v, w) \in E\}.$$

Then, for each vertex $v \in V \setminus B$, a reasonable criterion for parameterization quality is that v should be located at the center of its neighbors, illustrated in Figure 4.5(c). Mathematically, this condition is written

$$\vec{t}(v) = \frac{1}{|N(v)|} \sum_{w \in N(v)} \vec{t}(w).$$

Using this expression, we can associate each $v \in V$ with a linear condition either fixing its position on the boundary or asking that its assigned position equals the average of its neighbors' positions. This $|V| \times |V|$ system of equations defines a *harmonic* parameterization.

The final output in Figure 4.5(b) is laid out elastically, evenly distributing vertices on the image plane. Harmonic parameterization has been extended in countless ways to enhance the quality of this result, most prominently by accounting for the lengths of the edges in E as they are realized in three-dimensional space.

4.2 SPECIAL PROPERTIES OF LINEAR SYSTEMS

The examples above provide several contexts in which linear systems of equations are used to model practical computing problems. As derived in the previous chapter, Gaussian elimination solves all of these problems in polynomial time, but it remains to be seen whether this is the fastest or most stable technique. With this question in mind, here we look more closely at the matrices from §4.1 to reveal that they have many properties in common. By deriving solution techniques specific to these classes of matrices, we will design specialized algorithms with better speed and numerical quality.

4.2.1 Positive Definite Matrices and the Cholesky Factorization

As shown in Theorem 4.1, solving the least-squares problem $A\vec{x} \approx \vec{b}$ yields a solution $\vec{x}$ satisfying the square linear system $(A^\top A)\vec{x} = A^\top \vec{b}$. Regardless of A , the matrix $A^\top A$ has a few special properties that distinguish it from arbitrary matrices.

First, $A^\top A$ is symmetric, since by the identities $(AB)^\top = B^\top A^\top$ and $(A^\top)^\top = A$,

$$(A^\top A)^\top = A^\top (A^\top)^\top = A^\top A.$$

We can express this symmetry index-wise by writing $(A^\top A)_{ij} = (A^\top A)_{ji}$ for all indices i, j . This property implies that it is sufficient to store only the values of $A^\top A$ on or above the diagonal, since the rest of the elements can be obtained by symmetry.

Furthermore, $A^\top A$ is a *positive semidefinite* matrix, defined below:

Definition 4.1 (Positive (Semi-)Definite). A matrix $B \in \mathbb{R}^{n \times n}$ is positive semidefinite if for all $\vec{x} \in \mathbb{R}^n$, $\vec{x}^\top B \vec{x} \geq 0$. B is positive definite if $\vec{x}^\top B \vec{x} > 0$ whenever $\vec{x} \neq \vec{0}$.

The following proposition relates this definition to the matrix $A^\top A$:

Proposition 4.1. For any $A \in \mathbb{R}^{m \times n}$, the matrix $A^\top A$ is positive semidefinite. Furthermore, $A^\top A$ is positive definite exactly when the columns of A are linearly independent.

Proof. We first check that $A^\top A$ is *always* positive semidefinite. Take any $\vec{x} \in \mathbb{R}^n$. Then,

$$\vec{x}^\top (A^\top A) \vec{x} = (A\vec{x})^\top (A\vec{x}) = (A\vec{x}) \cdot (A\vec{x}) = \|A\vec{x}\|_2^2 \geq 0.$$

To prove the second statement, first suppose the columns of A are linearly independent. If A were only semidefinite, then there would be an $\vec{x} \neq \vec{0}$ with $\vec{x}^\top A^\top A \vec{x} = 0$, but as shown above, this would imply $\|A\vec{x}\|_2 = 0$, or equivalently $A\vec{x} = \vec{0}$, contradicting the independence of the columns of A . Conversely, if A has linearly *dependent* columns, then there exists a $\vec{y} \neq \vec{0}$ with $A\vec{y} = \vec{0}$. In this case, $\vec{y}^\top A^\top A \vec{y} = \vec{0}^\top \vec{0} = 0$, and hence A is not positive definite. $\square$

As a corollary, $A^\top A$ is invertible exactly when A has linearly independent columns, providing a condition to check whether a least-squares problem admits a unique solution.

Given the prevalence of the least-squares system $A^\top A \vec{x} = A^\top \vec{b}$, it is worth considering the possibility of writing faster linear solvers specially designed for this case. In particular, suppose we wish to solve a *symmetric positive definite* (SPD) system $C\vec{x} = \vec{d}$. For least-squares, we could take $C = A^\top A$ and $\vec{d} = A^\top \vec{b}$, but there also exist many systems that naturally are symmetric and positive definite without explicitly coming from a least-squares model. We could solve the system using Gaussian elimination or LU factorization, but given the additional structure on C we can do somewhat better.

Aside 4.1 (Block matrix notation). Our construction in this section will rely on *block matrix* notation. This notation builds larger matrices out of smaller ones. For example, suppose $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{m \times k}$, $C \in \mathbb{R}^{p \times n}$, and $D \in \mathbb{R}^{p \times k}$. Then, we could construct a larger matrix by writing:

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \in \mathbb{R}^{(m+p) \times (n+k)}.$$

This “block matrix” is constructed by concatenation. Block matrix notation is convenient, but we must be careful to concatenate matrices with dimensions that match. The mechanisms of matrix algebra generally extend to this case, e.g.,

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} E & F \\ G & H \end{pmatrix} = \begin{pmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{pmatrix}.$$

We will proceed without checking these identities explicitly, but as an exercise it is worth double-checking that they are true.

We can deconstruct the symmetric positive-definite matrix $C \in \mathbb{R}^{n \times n}$ as a block matrix:

$$C = \begin{pmatrix} c_{11} & \vec{v}^\top \\ \vec{v} & \tilde{C} \end{pmatrix}$$

where $c_{11} \in \mathbb{R}$, $\vec{v} \in \mathbb{R}^{n-1}$, and $\tilde{C} \in \mathbb{R}^{(n-1) \times (n-1)}$. The SPD structure of C provides the following observation:

$$\begin{aligned} 0 &< \vec{e}_1^\top C \vec{e}_1 \text{ since } C \text{ is positive definite and } \vec{e}_1 \neq \vec{0} \\ &= \begin{pmatrix} 1 & 0 & \cdots & 0 \end{pmatrix} \begin{pmatrix} c_{11} & \vec{v}^\top \\ \vec{v} & \tilde{C} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \end{aligned}$$

$$\begin{aligned}
&= \begin{pmatrix} 1 & 0 & \cdots & 0 \end{pmatrix} \begin{pmatrix} c_{11} \\ \vec{v} \end{pmatrix} \\
&= c_{11}.
\end{aligned}$$

By the strict inequality in the first line, we do not have to use pivoting to guarantee that $c_{11} \neq 0$ in the first step of Gaussian elimination.

Continuing with Gaussian elimination, we can apply a forward-substitution matrix E of the form

$$E = \begin{pmatrix} 1/\sqrt{c_{11}} & \vec{0}^\top \\ \vec{r} & I_{(n-1) \times (n-1)} \end{pmatrix}.$$

Here, the vector $\vec{r} \in \mathbb{R}^{n-1}$ contains forward-substitution scaling factors satisfying $r_{i-1}c_{11} = -c_{i1}$. Unlike our original construction of Gaussian elimination, we scale row 1 by $1/\sqrt{c_{11}}$ for reasons that will become apparent shortly.

By design, after forward-substitution, the form of the product EC is:

$$EC = \begin{pmatrix} \sqrt{c_{11}} & \vec{v}^\top/\sqrt{c_{11}} \\ \vec{0} & D \end{pmatrix},$$

for some $D \in \mathbb{R}^{(n-1) \times (n-1)}$.

Now, we diverge from the derivation of Gaussian elimination. Rather than moving on to the second row, to maintain symmetry, we post-multiply by $E^\top$ to obtain $ECE^\top$:

$$\begin{aligned}
ECE^\top &= (EC)E^\top \\
&= \begin{pmatrix} \sqrt{c_{11}} & \vec{v}^\top/\sqrt{c_{11}} \\ \vec{0} & D \end{pmatrix} \begin{pmatrix} 1/\sqrt{c_{11}} & \vec{r}^\top \\ \vec{0} & I_{(n-1) \times (n-1)} \end{pmatrix} \\
&= \begin{pmatrix} 1 & \vec{0}^\top \\ \vec{0} & D \end{pmatrix}.
\end{aligned}$$

The $\vec{0}^\top$ in the upper right follows from the construction of E as an elimination matrix. Alternatively, an easier if less direct argument is that $ECE^\top$ is symmetric, and the lower-left element of the block form for $ECE^\top$ is $\vec{0}$ by block matrix multiplication. Regardless, we have eliminated the first row *and* the first column of C ! Furthermore, the remaining submatrix D is also symmetric and positive definite, as suggested in Exercise 4.2.

Example 4.6 (Cholesky factorization, initial step). As a concrete example, consider the following symmetric, positive definite matrix

$$C = \begin{pmatrix} 4 & -2 & 4 \\ -2 & 5 & -4 \\ 4 & -4 & 14 \end{pmatrix}.$$

We can eliminate the first column of C using the elimination matrix E_1 defined as:

$$E_1 = \begin{pmatrix} 1/2 & 0 & 0 \\ 1/2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix} \longrightarrow E_1 C = \begin{pmatrix} 2 & -1 & 2 \\ 0 & 4 & -2 \\ 0 & -2 & 10 \end{pmatrix}.$$

We chose the upper left element of E_1 to be $1/2 = 1/\sqrt{4} = 1/\sqrt{c_{11}}$. Following the construction above, we can post-multiply by $E_1^\top$ to obtain:

$$E_1 C E_1^\top = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 4 & -2 \\ 0 & -2 & 10 \end{pmatrix}.$$

The first row *and* column of this product equal the standard basis vector $\vec{e}_1 = (1, 0, 0)$.

We can repeat this process to eliminate all the rows and columns of C symmetrically. This method is *specific* to symmetric positive-definite matrices, since

- symmetry allowed us to apply the same E to both sides, and
- positive definiteness guaranteed that $c_{11} > 0$, thus implying that $1/\sqrt{c_{11}}$ exists.

Similar to LU factorization, we now obtain a factorization $C = LL^\top$ for a lower-triangular matrix L . This factorization is constructed by applying elimination matrices symmetrically using the process above, until we reach

$$E_k \cdots E_2 E_1 C E_1^\top E_2^\top \cdots E_k^\top = I_{n \times n}.$$

Then, like our construction in §3.5.1, we define L as a product of lower-triangular matrices:

$$L \equiv E_1^{-1} E_2^{-1} \cdots E_k^{-1}.$$

The product $C = LL^\top$ is known as the *Cholesky factorization* of C . If taking the square roots causes numerical issues, a related $LDL^\top$ factorization, where D is a diagonal matrix, avoids this issue and can be derived from the discussion above; see Exercise 4.6.

Example 4.7 (Cholesky factorization, remaining steps). Continuing Example 4.6, we can eliminate the second row and column as follows:

$$E_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1/2 & 0 \\ 0 & 1/2 & 1 \end{pmatrix} \rightarrow E_2(E_1 C E_1^\top) E_2^\top = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 9 \end{pmatrix}.$$

Rescaling brings the symmetric product to the identity matrix $I_{3 \times 3}$:

$$E_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1/3 \end{pmatrix} \rightarrow E_3(E_2 E_1 C E_1^\top E_2^\top) E_3^\top = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Hence, we have shown $E_3 E_2 E_1 C E_1^\top E_2^\top E_3^\top = I_{3 \times 3}$. As above, define:

$$L = E_1^{-1} E_2^{-1} E_3^{-1} = \begin{pmatrix} 2 & 0 & 0 \\ -1 & 1 & 0 \\ 2 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 3 \end{pmatrix} = \begin{pmatrix} 2 & 0 & 0 \\ -1 & 2 & 0 \\ 2 & -1 & 3 \end{pmatrix}.$$

This matrix L satisfies $LL^\top = C$.

The Cholesky factorization has many practical properties. It takes half the memory to store L from the Cholesky factorization rather than the LU factorization of C . Specifically, L has $n(n+1)/2$ nonzero elements, while the compressed storage of LU factorizations explained in §3.5.3 requires n^2 nonzeros. Furthermore, as with the LU decomposition, solving $C\vec{x} = \vec{d}$ can be accomplished using fast forward- and back-substitution. Finally, the product $LL^\top$ is symmetric and positive semidefinite regardless of L ; if we factored $C = LU$ but made rounding and other mistakes, in degenerate cases the computed product $C' \approx LU$ may no longer satisfy these criteria exactly.

Code for Cholesky factorization can be very succinct. To derive a particularly compact form, we can work backward from the factorization $C = LL^\top$ now that we know such an

object exists. Suppose we choose an arbitrary $k \in \{1, \dots, n\}$ and write L in block form isolating the k -th row and column:

$$L = \begin{pmatrix} L_{11} & \vec{0} & 0 \\ \vec{\ell}_k^\top & \ell_{kk} & \vec{0}^\top \\ L_{31} & \vec{\ell}_k' & L_{33} \end{pmatrix}.$$

Here, since L is lower triangular, L_{11} and L_{33} are both lower-triangular square matrices. Applying block matrix algebra to the product $C = LL^\top$ shows:

$$\begin{aligned} C = LL^\top &= \begin{pmatrix} L_{11} & \vec{0} & 0 \\ \vec{\ell}_k^\top & \ell_{kk} & \vec{0}^\top \\ L_{31} & \vec{\ell}_k' & L_{33} \end{pmatrix} \begin{pmatrix} L_{11}^\top & \vec{\ell}_k & L_{31}^\top \\ \vec{0}^\top & \ell_{kk} & (\vec{\ell}_k')^\top \\ 0 & \vec{0} & L_{33}^\top \end{pmatrix} \\ &= \begin{pmatrix} \times & \times & \times \\ \vec{\ell}_k^\top L_{11}^\top & \vec{\ell}_k^\top \vec{\ell}_k + \ell_{kk}^2 & \times \\ \times & \times & \times \end{pmatrix}. \end{aligned}$$

We leave out values of the product that are not necessary for our derivation.

Since $C = LL^\top$, from the product above we now have $c_{kk} = \vec{\ell}_k^\top \vec{\ell}_k + \ell_{kk}^2$, or equivalently

$$\ell_{kk} = \sqrt{c_{kk} - \|\vec{\ell}_k\|_2^2},$$

where $\vec{\ell}_k \in \mathbb{R}^{k-1}$ contains the elements of the k -th row of L to the left of the diagonal. We can choose $\ell_{kk} \geq 0$, since scaling columns of L by -1 has no effect on $C = LL^\top$. Furthermore, applying $C = LL^\top$ to the middle left element of the product shows $L_{11}\vec{\ell}_k = \vec{c}_k$, where $\vec{c}_k$ contains the elements of C in the same position as $\vec{\ell}_k$. Since L_{11} is lower triangular, this system can be solved by forward-substitution for $\vec{\ell}_k$!

Synthesizing the formulas above reveals an algorithm for computing the Cholesky factorization by iterating $k = 1, 2, \dots, n$. L_{11} will already be computed by the time we reach row k , so $\vec{\ell}_k$ can be found using forward-substitution. Then, ℓ_{kk} is computed directly using the square root formula. We provide pseudocode in Figure 4.6. As with LU factorization, this algorithm runs in $O(n^3)$ time; more specifically, Cholesky factorization takes approximately $\frac{1}{3}n^3$ operations, half the work needed for LU.

4.2.2 Sparsity

We set out in this section to identify properties of specific linear systems that can make them solvable using more efficient techniques than Gaussian elimination. In addition to positive definiteness, many linear systems of equations naturally enjoy *sparsity*, meaning that most of the entries of A in the system $A\vec{x} = \vec{b}$ are exactly zero. Sparsity can reflect particular structure in a given problem, including the following use cases:

- In image processing (e.g., §4.1.5), systems for photo editing express relationships between the values of pixels and those of their neighbors on the image grid. An image may be a point in $\mathbb{R}^p$ for p pixels, but when solving $A\vec{x} = \vec{b}$ for a new size- p image, $A \in \mathbb{R}^{p \times p}$ may have only $O(p)$ rather than $O(p^2)$ nonzeros since each row only involves a single pixel and its up/down/left/right neighbors.
- In computational geometry (e.g., §4.1.6), shapes are often expressed using collections of triangles linked together into a mesh. Equations for surface smoothing, parameterization, and other tasks link values associated with given vertex with only those at their neighbors in the mesh.

```

function CHOLESKY-FACTORIZATION( $C$ )
  ▷ Factors  $C = LL^T$ , assuming  $C$  is symmetric and positive definite
   $L \leftarrow C$                                 ▷ This algorithm destructively replaces  $C$  with  $L$ 
  for  $k \leftarrow 1, 2, \dots, n$ 
    ▷ Back-substitute to place  $\vec{\ell}_k^T$  at the beginning of row  $k$ 
    for  $i \leftarrow 1, \dots, k-1$                 ▷ Current element  $i$  of  $\vec{\ell}_k$ 
       $s \leftarrow 0$ 
      ▷ Iterate over  $L_{11}$ ;  $j < i$ , so the iteration maintains  $L_{kj} = (\vec{\ell}_k)_j$ .
      for  $j \leftarrow 1, \dots, i-1 : s \leftarrow s + L_{ij}L_{kj}$ 
       $L_{ki} \leftarrow (L_{ki} - s) / L_{ii}$ 

    ▷ Apply the formula for  $\ell_{kk}$ 
     $v \leftarrow 0$                                 ▷ For computing  $\|\vec{\ell}_k\|_2^2$ 
    for  $j \leftarrow 1, \dots, k-1 : v \leftarrow v + L_{kj}^2$ 
     $L_{kk} \leftarrow \sqrt{L_{kk} - v}$ 
  return  $L$ 

```

Figure 4.6 Cholesky factorization for writing $C = LL^T$, where the input C is symmetric and positive-definite and the output L is lower triangular.

- In machine learning, a *graphical model* uses a graph $G = (V, E)$ to express probability distributions over several variables. Each variable corresponds to a node $v \in V$, and edges $e \in E$ represent probabilistic dependences. Linear systems in this context often have one row per $v \in V$ with nonzeros in columns involving v and its neighbors.

If $A \in \mathbb{R}^{n \times n}$ is sparse to the point that it contains $O(n)$ rather than $O(n^2)$ nonzero values, there is no reason to store A with n^2 values. Instead, *sparse matrix* storage techniques only store the $O(n)$ nonzeros in a more reasonable data structure, e.g., a list of row/column/value triplets. The choice of a matrix data structure involves considering the likely operations that will occur on the matrix, possibly including multiplication, iteration over nonzeros, or iterating over individual rows or columns.

Unfortunately, the LU (and Cholesky) factorizations of a sparse matrix A may not result in sparse L and U matrices; this loss of structure severely limits the applicability of using these methods to solve $A\vec{x} = \vec{b}$ when A is large but sparse. Thankfully, there are many *direct sparse solvers* that produce an LU-like factorization without inducing much *fill*, or additional nonzeros; discussion of these techniques can be found in [32]. Alternatively, *iterative* techniques can obtain approximate solutions to linear systems using only multiplication by A and A^T . We will derive some of these methods in Chapter 11.

4.2.3 Additional Special Structures

Certain matrices are not only sparse but also *structured*. For instance, a *tridiagonal* system of linear equations has the following pattern of nonzero values:

$$\begin{pmatrix} \times & \times & & & \\ \times & \times & \times & & \\ & \times & \times & \times & \\ & & \times & \times & \times \\ & & & \times & \times \end{pmatrix}.$$